TradeMirror — Post-Trade Review Assistant

Pranaykumar Akuthota · Problem 03 · 547 words including headings

The problem

Nubra’s post-trade review brief asks us to turn a month of order history into two or three habits costing a trader money. The difficult part is not calculating losses: it is deciding what those records can responsibly tell us. Order history does not reveal intent, emotions, strategy quality, or what would have happened under a different decision.

TradeMirror targets a self-directed intraday equity trader who wants a short monthly review without maintaining an elaborate journal. This is a proposed user segment, not a finding from interviews. The product helps them understand what happened, investigate possible patterns, and choose one measurable learning experiment.

Why Nubra could solve it

A trading platform can connect execution history with reflection in the same workflow. A compact review could provide value beyond placing the next order. Whether this increases review frequency, retention, or trading quality remains unvalidated; this prototype makes no claims about those outcomes.

Solution and scope

The solution space includes full trading journals, metric-heavy dashboards, and conversational assistants. TradeMirror chooses a short evidence-led review: less bookkeeping than a journal, fewer distractions than a dashboard, and explanations that can be checked directly against records rather than generated assertions.

TradeMirror presents a clearly labeled synthetic month, a factual summary, up to two observations, exact supporting records, and one saved experiment. Calculations are deterministic and local. No external model produces amounts or infers behavior.

The exploratory observation compares same-instrument, same-day re-entry trades with classifiable non-re-entry trades. These groups are mutually exclusive but not matched controls. Differences may reflect position size, instrument mix, strategy, or market conditions. The product checks whether excluding the largest losing re-entry changes the result. If that makes the result misleading or leaves insufficient recurrence, it withholds the card and explains why. Operational minima are disclosed as product guardrails, not statistical confidence.

A separate observation shows entries beyond a fictional trader’s explicitly stated daily limit. The limit is additional scenario context, not something inferred from the records. Profitable deviations remain deviations. Their outcomes cannot establish whether breaking the plan caused a gain or loss.

Each observation opens its definitions, supporting trades, comparisons, calculations, and limitations. The user then chooses a record-keeping experiment, such as recording a reason before each re-entry during the next ten completed trades. This measures documentation, not improved returns. The selection is saved only in the browser, with a visible fallback if storage is unavailable.

Boundaries and evaluation

The prototype starts with normalized, fully closed intraday long-equity positions. It does not solve broker import, raw-order matching, partial exits, derivatives, overnight trades, or live risk. No backend, brokerage connection, trading recommendation, or savings estimate is included.

Evaluation focuses on exact arithmetic, costs, duplicate and invalid records, IST date boundaries, evidence reconciliation, deliberate abstention, sensitivity reversal, and the complete review-to-experiment journey. Automated calculation and Chromium tests passed locally, including the production bundle. These results establish implementation behavior, not user demand or financial benefit. The prototype is hosted on Netlify; checks on the prior deployment and the updated deployment’s verification limits are documented in the evals. No usability research has been conducted.

The central product decision is simple: correct arithmetic does not make a behavioral conclusion reliable. A useful review must show both the evidence and its limits.

The prompts used

The following task prompts are reproduced from the preserved prompt files. They include the requests that challenged and replaced the original heuristic rules. Line wrapping is for readability; unmodified UTF-8 source files are also embedded as PDF attachments. This is a task-prompt record, not a complete assistant/tool transcript. Screenshot-only messages and routine status replies are not reproduced.

AI assisted framing, critique, implementation, tests and copy. The app itself uses deterministic local code, not a runtime LLM. Failed attempts and corrections follow the prompts.

01-original-request.txt

```
I am working on a real take-home assignment for Nubra's Product Management Intern selection process. I want you to act as my senior product-engineering collaborator: help me reason through the problem, challenge weak assumptions, and build a reliable, thoughtful product that I can personally understand and defend in an interview.

I have attached three documents:
1. Nubra Product Intern Assignment PDF — the primary source of truth for the assignment requirements.
2. Job Description PDF — context about the Product Management Intern role.
3. My submitted resume — context about my actual skills and experience.

Read all three documents before proposing a solution. Respect the assignment's exact requirements and distinguish explicit requirements from your own recommendations.

## DECISION ALREADY MADE

I have selected Problem 03: Post-trade review.

The assignment describes the problem as:
"Most retail traders never review their own trades. Turn a month of order history into the two or three habits that are costing them money."

Do not switch to another problem unless you identify a substantial, evidence-based reason and explain it to me first.

## MY OBJECTIVE

Build a clickable, working MVP that demonstrates strong product judgment, clear UX, reliable calculations, meaningful testing, and thoughtful use of AI.

I want the result to feel like a carefully reasoned product built by me with AI assistance—not a generic AI-generated dashboard. Do not optimize merely for visual appearance or the number of features.

I am aiming for the strongest possible submission, while understanding that selection cannot be guaranteed.

## PREFERRED TECHNOLOGY

- React
- Vite
- JavaScript
- Tailwind CSS
- Recharts only if it provides genuine value
- Local, clearly labeled synthetic trade data for the initial prototype
- Git for version control
- Automated tests for important calculations and user flows

Keep the architecture simple. Do not introduce a backend, database, authentication, live brokerage integration, paid API, or external LLM dependency unless you establish a clear MVP need and explain the trade-off first.

## PRODUCT DIRECTION

The provisional product concept is a post-trade review assistant that turns historical trades into a small number of evidence-backed observations and helps a trader choose one measurable habit to test.

This is a starting hypothesis, not a fixed feature specification. Critically assess it.

Potential capabilities to evaluate:
- Import or inspect a sample month of trade history.
- Summarize realized P&L, trade count, win rate, and other metrics only when the data supports them.
- Identify a maximum of two or three recurring behavioral patterns.
- Show the trades, calculations, sample size, and limitations supporting each pattern.
- Let the user inspect why an insight was generated.
- Suggest a measurable review habit without issuing personalized buy/sell recommendations.
- Handle insufficient data, invalid records, missing fields, and zero-trade periods honestly.

Do not automatically implement every item. Select the smallest coherent set that solves the core problem well.
```

NON-NEGOTIABLE PRODUCT PRINCIPLES

1. Numerical metrics must be calculated deterministically in application code, not invented by an LLM.
2. Every generated insight must be traceable to the underlying trade records.
3. Distinguish observed patterns from causal claims. Do not claim a behavior caused losses unless the evidence actually supports that conclusion.
4. Show sample sizes and meaningful limitations.
5. Do not manufacture trading statistics, research findings, user interviews, or validation results.
6. If the evidence is weak, the application must say so rather than forcing an insight.
7. Clearly label synthetic data and disclose assumptions such as how P&L and transaction costs are represented.
8. Keep the experience understandable to a retail trader who is not a financial expert.
9. Every primary interaction must work; do not use decorative controls that appear functional but do nothing.
10. Avoid unnecessary complexity, excessive charts, generic AI chat interfaces, and unsupported financial advice.

HOW I WANT YOU TO WORK

We will work in small, reviewable stages.

STAGE 1 — UNDERSTAND AND PLAN. Do not write application code yet.

First:
A. Summarize the assignment's exact deliverables, time budget, and constraints from the attached assignment PDF.
B. Extract the relevant responsibilities and expectations from the Product Management Intern job description.
C. Identify the resume skills and project experience relevant to this particular assignment. Do not exaggerate my experience.
D. Compare the proposed product direction with the exact wording of Problem 03.
E. Define the target user, problem statement, primary user job, key assumptions, and the most important unanswered questions.
F. Propose one core user journey from opening the app to understanding an insight and choosing a measurable habit.
G. Define the MVP scope and explicitly list what we will not build.
H. Propose the trade data schema and metric definitions, including how to handle partial exits, missing costs, duplicate records, and insufficient samples. Flag any assumptions that need to be simplified for the MVP.
I. Explain which calculations should be deterministic and which, if any, genuinely benefit from AI.
J. Propose an initial component architecture, an implementation sequence, and a realistic testing strategy.
K. Identify the main ways this product could mislead users and how we will prevent them.
L. Define measurable acceptance criteria for the finished prototype.

REQUIRED RESPONSE FORMAT

Present your findings in these sections:
1. Assignment requirements
2. Relevant candidate strengths
3. Problem framing and target user
4. Proposed product experience
5. MVP scope and exclusions
6. Data model and metric definitions
7. Technical architecture
8. Evaluation plan and acceptance criteria
9. Key risks, assumptions, and trade-offs
10. A prioritized implementation plan

For each significant recommendation, briefly explain why it is appropriate for this assignment.

Be specific and critical. Avoid generic product-management advice and unnecessary jargon.

Do not generate the one-pager, write application code, create files, install dependencies, or deploy anything during this first stage.

End with the three most important decisions you need me to approve before implementation.

I will review your plan and give feedback. Only then should we proceed to implementation.

02-independent-critique-request.txt

```
Before we approve the implementation plan, I want you to independently evaluate whether choosing
Problem 03 — Post-trade Review is actually the strongest decision for this Nubra Product Management
Intern assignment.

Do not simply defend your existing recommendation or agree with me. Act as a skeptical senior product
manager reviewing a candidate's take-home assignment.

Compare all four original problem statements from the attached Nubra assignment PDF:

1. Stock Fundamental Research Assistant
2. Position Risk Assistant
3. Post-trade Review
4. First F&O Trade

Evaluate each problem against these criteria:

- Clarity and importance of the user problem
- Ability to demonstrate strong product thinking rather than just UI development
- Ability to build a convincing, clickable MVP within the assignment's 4–6-hour suggested effort
- Ability to validate the solution with credible, reproducible tests
- Potential for meaningful differentiation from generic AI-generated prototypes
- Relevance to Nubra's trading platform and the Product Management Intern responsibilities
- Risk of incorrect financial reasoning, unsupported claims, or misleading users
- Ability to explain and defend the product decisions in a follow-up interview

For each criterion, give a score from 1 to 5 and explain the most important reason behind the score.
Define the scoring scale and use the same standard for all four problems.

Then answer these questions directly:

1. If you were evaluating this candidate, which of the four submissions would most likely make you want
to interview the candidate further, and why?
2. Is Problem 03 genuinely the strongest option, or is it merely the easiest to implement?
3. What is the strongest argument AGAINST choosing Problem 03?
4. What would Problems 02 or 04 allow us to demonstrate that Problem 03 cannot?
5. What specific product or technical insight could make a Problem 03 submission distinctive rather
than generic?
6. What evidence would change your recommendation to another problem?
7. Given my submitted resume and the Product Management Intern job description, which problem offers
the best opportunity to demonstrate my strengths without making my submission look like a backend
engineering project?
8. Could our proposed deterministic pattern rules be too arbitrary or misleading? Critique the
later-entry and rapid-re-entry rules, including the sample-size thresholds and time cutoffs. Suggest a
safer or more defensible alternative if needed.

Important constraints:

- Use the actual four problem statements in the assignment as your primary source.
- Distinguish facts stated in the assignment from your own inferences.
- Do not pretend to know Nubra's internal evaluation rubric or predict a guaranteed selection.
- Do not change the chosen problem just to make the response more interesting.
- If you recommend staying with Problem 03, explain why it is better than the alternatives beyond
implementation convenience.
- If you recommend switching, provide a concrete argument showing why the alternative would produce a
materially stronger submission within the time limit.

Finish with:\
A. Your recommended problem and confidence level.\
B. The three biggest trade-offs of that choice.\
C. Whether our current MVP plan should remain unchanged, be revised, or be reconsidered before
implementation.

Do not write application code or begin implementation. I want this independent critique so I can make
an informed final decision.
```

03-revised-specification-request.txt

```
We have decided to stay with **Problem 03 — Post-trade Review**, but revise the MVP based on your
critique. Do not start coding yet.

Your central insight was: “Correct arithmetic does not make a behavioral conclusion reliable.” Make
that principle a core product requirement.

### 1. Finalize the product concept

Keep the working name **TradeMirror — Post-Trade Review Assistant** unless you have a concrete reason
to change it.

The product should help a retail trader review one month of completed trades, investigate possible
patterns, inspect the supporting records, understand uncertainty, and choose one measurable experiment
for their next trading period.

It must not pretend that order history reveals intent, emotions, strategy quality, causation, or what
would have happened under a different decision.

### 2. Revise the insight engine

- Remove “fourth trade onward” as a default costly-habit detector.
- Do not label rapid re-entry as revenge trading or automatically recommend waiting 30 minutes.
- Separate **observed facts**, **exploratory patterns**, and **user-stated-plan deviations**.
- If a user-stated trading plan is supported by the available data, show deviations from it without
implying that the deviation caused a loss.
- If the dataset cannot support a useful pattern, show a clear abstention state rather than
manufacturing insights.
- Do not use arbitrary sample thresholds to imply statistical significance or confidence.
- Include a sensitivity check where appropriate, such as showing whether an observed result changes
when the largest losing trade is excluded.
- Never claim that an identified amount is avoidable loss or money the user could have saved.

### 3. Keep the MVP feasible

We have only 4–6 hours. Use synthetic, clearly disclosed, fully closed intraday long-equity trade
records. Keep the analysis deterministic and local, with no broker integration, backend, external LLM,
or live financial data.

Design a short journey:

1. Open a clearly labeled synthetic review.
2. See the month's factual summary.
3. Explore zero, one, or two evidence-supported observations.
4. Inspect the exact trades, definitions, comparisons, and limitations.
5. Choose one measurable review or record-keeping experiment for the next period.

A useful experiment might be recording the reason for each re-entry for the next ten trades. Present
this as a way to learn about the user's own behavior, not a proven method for improving returns.

### 4. Specify the evidence model

For each proposed observation, define:

- What is directly observed.
- How the relevant trades are selected.
- What comparison, if any, is used and why.
- Which records support the result.
- What could make the result misleading.
- Whether a sensitivity check is relevant.
- What the product should display if the result is inconclusive.

Do not invent a statistically justified threshold. Explain the rationale for any operational minimum
you retain, and label it as a product guardrail rather than statistical proof.

### 5. Define realistic evaluations

Create a compact, reproducible test plan covering:

- Exact P&L arithmetic and transaction costs.
```

- Invalid, duplicated, conflicting, and insufficient records.
- Boundary timestamps and review-month assignment.
- Evidence rows reconciling with displayed figures.
- Datasets where the tool must abstain.
- A result that reverses when its largest losing trade is excluded.
- A case where plan deviation is observed but causation cannot be claimed.
- The complete review-to-experiment journey and local persistence.

Distinguish automated tests from manual review. Do not claim user research, usability testing, or successful deployment unless actually performed.

6. Keep the assignment deliverables in view

The final submission must contain:

1. A 300–700-word product one-pager.
2. Actual prompts used, as-is, including failed attempts.
3. Evals used and how we decided the prototype was good enough.
4. A working, accessible app link.

Keep the product-management decisions more prominent than the implementation details. The submission should demonstrate problem framing, scope judgment, trust, and evaluation—not simply a polished dashboard.

Deliver your response

Return:\
A. Final product thesis and target user.\
B. Exact MVP scope and explicit exclusions.\
C. Screen-by-screen user journey.\
D. Proposed data schema and calculation definitions.\
E. Observation types you will implement, with clear justification.\
F. Edge cases and abstention behavior.\
G. Concrete acceptance criteria and test cases.\
H. Revised 4–6-hour implementation plan.\
I. Main unresolved decisions or risks.

Challenge your own plan where needed. Make the specification internally consistent and achievable before recommending implementation. Do not write code yet.

04-implementation-request.txt

```
The revised TradeMirror specification is approved, with two clarifications. Start implementation now.

### Clarifications before coding

1. **Re-entry comparison:** Treat this as an exploratory descriptive comparison only. Clearly define
mutually exclusive groups: qualifying same-instrument, same-day re-entry trades versus classifiable
non-re-entry trades. Do not describe the comparison as causal, matched, statistically significant, or
proof of a costly habit. If the data quality or sensitivity check makes the result misleading, withhold
the card and explain why.
2. **Date semantics:** Use IST entry date for trading-day sequence and stated daily-entry-limit
calculations. Use IST close date for review-month inclusion and monthly summary. Document this
distinction and test month-boundary cases. All timestamps must be normalized consistently before
comparing them.

### Implementation requirements

Build **TradeMirror — Post-Trade Review Assistant** according to the approved specification.

- First inspect the existing project, attached assignment, job description, and resume. Reuse the
current project structure if appropriate; do not overwrite useful work without checking.
- Use React, Vite, JavaScript, and Tailwind CSS if they fit the existing setup.
- Keep all calculations and observation rules deterministic, separated from UI components, and testable.
- Use bundled synthetic data, clearly labeled throughout.
- Implement the factual monthly summary, zero-to-two observation cards, evidence inspection, complete
trade ledger, exclusion reasons, sensitivity check, abstention states, and one saved learning
experiment.
- Implement the limited-data example so the reviewer can see the product deliberately withhold an
unsupported observation.
- Use Indian rupee formatting and integer paise for internal monetary arithmetic.
- Make every displayed figure reconcile with its underlying evidence.
- Include responsive layout, accessible labels, keyboard-operable controls, meaningful
loading/error/empty states, and a visible browser-persistence fallback.
- Do not add a backend, broker integration, live data, external LLM, unsupported analytics, or
unnecessary features.

### Test before polishing

Write and run the calculation and validation tests first. Include the specification's hand-calculated
arithmetic, duplicates/conflicts, invalid records, IST boundaries, sequence ambiguity, evidence
reconciliation, insufficient samples, sensitivity reversal, profitable plan deviations, and abstention
cases.

Then build the main user journey and run the available automated tests. Fix failures instead of hiding
or weakening tests. Do not claim that a test passed unless it actually ran and passed.

### Delivery

1. Implement the app in the current project.
2. Run the test suite and production build.
3. Report the exact commands run, results, remaining failures, and known limitations.
4. Summarize the files created or changed and how to run the project locally.
5. Provide a short manual QA checklist for the remaining browser checks.
6. Preserve the actual prompts and failed attempts for the assignment's prompt-history artifact.

Do not claim deployment or public accessibility until verified. If deployment credentials or hosting
access are unavailable, prepare the production build and give me the precise next steps to publish it.

**Priority order:** correctness → evidence and abstention → complete user journey → accessibility and
responsive behavior → visual polish.

Begin by inspecting the repository and then implement. Do not return another proposal instead of doing
the work.
```

05-predeployment-audit-request.txt

```
Perform a final pre-deployment audit of the implemented TradeMirror repository at
`/workspace/Nubra-trademirror`. Do not add features or redesign the application.

Read the actual source code, tests, README, `docs/prompt-history.md`, `docs/evals.md`, and
`docs/product-one-pager.md`. Compare them against the approved specification and the Nubra Product
Management Intern assignment.

### Audit the application

1. Verify that the visible synthetic-data disclosures are clear.
2. Check every displayed metric against the calculation engine and its supporting records.
3. Verify that re-entry comparisons are mutually exclusive, descriptive only, and not presented as
causal findings.
4. Verify that the largest-loss sensitivity check and abstention behavior are implemented as specified.
5. Verify that stated-plan deviations do not imply causation, avoidable losses, or bad trading behavior.
6. Check date handling, invalid and conflicting records, exclusion explanations, and local experiment
persistence.
7. Check for dead buttons, misleading labels, broken evidence panels, and responsive-layout issues.
8. Check accessibility basics, including keyboard operation, focus visibility, accessible names, and
semantic structure.

### Audit the submission artifacts

1. Confirm that the one-pager is between 300 and 700 words and covers the problem, why Nubra should
solve it, scope, exclusions, and solution space.
2. Confirm the prompt history contains actual prompts used and actual failed attempts. Do not fabricate
missing history.
3. Confirm the evaluation document distinguishes tests actually executed from proposed or manual checks
not yet completed.
4. Confirm that no document claims user research, usability validation, deployment, or product impact
that has not occurred.
5. Ensure the app is described as a prototype using synthetic data, not a production-ready trading
product.

### Verification

Run the relevant existing test commands and production build again. Fix only concrete defects within
the existing scope. Do not weaken tests to make them pass.

Return:

- Findings classified as blocker, important, or minor.
- Exact files and lines involved.
- Fixes made.
- Exact commands run and their actual results.
- Remaining manual checks.
- A final go/no-go recommendation for deployment.

Do not claim to have visually checked a browser, tested a screen reader, or verified public access
unless you actually performed that check. Do not deploy yet.
```

06-deployment-request.txt

```
Proceed with deployment of the audited TradeMirror prototype. The deployment is authorized.

Repository: `/workspace/Nubra-trademirror`

### Deployment plan

1. Inspect the README and existing project configuration for the intended static hosting setup.
2. Prefer Vercel if the environment is already authenticated and deployment is available. Otherwise,
use an already-configured static host if one is available.
3. Deploy the production Vite build, using `npm run build` and `dist` as the output directory.
4. Do not commit or push to a remote repository without checking the configured remote and obtaining
any needed authorization. Manual deployment is acceptable.
5. Never expose environment secrets, access tokens, or credentials in logs or output.

### After deployment

Verify the actual public URL:

- Open it without relying on the development server or an authenticated author session.
- Confirm the page loads and its JavaScript and CSS assets load successfully.
- Exercise the monthly summary, evidence panels, abstention example, and experiment selection.
- Reload the page and check browser-local persistence.
- Check a phone-sized viewport and horizontal evidence-table scrolling.
- Confirm no development-only paths or localhost dependencies break the hosted build.

Do not claim signed-out access or persistence was verified unless you actually tested it.

### Report back

Provide:

1. The exact public URL.
2. Hosting provider and deployment method.
3. Build and deployment results.
4. Which public-link checks actually passed.
5. Any outstanding limitations.

If authentication or hosting permissions are unavailable, stop and give me the shortest exact manual
deployment steps. Do not fabricate a URL or claim success before verification.
```

07-ux-refinement-request.txt

```
okay one thing i want to ask is , do everything like UI and UX which is react and javascript are
utilized full fledged or not , becuase i need this thing to get selected and outstand so make this
accordingly
```

08-ui-polish-request.txt

```
so add if you would like to improve UI if needed
```

09-final-quality-pass-request.txt

```
We are preparing the final submission of TradeMirror — Post-Trade Review Assistant for the Nubra
Product Management Intern assignment.

Project: `/workspace/Nubra-trademirror`

Latest Netlify URL shown in my screenshot:
`https://kaleidoscopic-basbousa-1e9c35.netlify.app`

The submitted PDF currently lists a different URL:
`https://sparkling-pixie-91ad77.netlify.app`

Your job is to perform a final, evidence-based quality pass. Prioritize correctness, reviewer
usability, product judgment, and submission credibility over adding features. Do not promise a perfect
score or claim selection is guaranteed.

## 1. Inspect before changing anything

Read the current source, tests, README, all relevant docs, and the actual submission artifact files if
they are present. Check the latest deployment instructions and the available Git status.

Do not overwrite or discard useful work. Do not assume the latest Netlify URL is accessible just
because it appears in a screenshot.

## 2. Resolve the deployment URL mismatch

- Verify which URL is intended to be the final deployment.
- If public access can be tested with available tools, test the candidate URL without relying on the
author's authenticated session.
- Check that the page, JavaScript, CSS, and core workflow load.
- Do not redeploy or change the public site unless necessary and authorized.
- Update the final submission PDF and any other submission artifact to use the confirmed final URL
consistently.
- If public access cannot be independently verified, explicitly mark it unverified. Do not invent
successful checks.

## 3. Fix concrete UX and responsive issues

The evaluation notes identify a Netlify badge overlapping content while scrolling on mobile.

- Determine whether the badge is part of the deployed app or injected by the hosting preview/platform.
- If it is part of our app, fix the overlap without hiding essential content or breaking navigation.
- Inspect the app at 375px width and desktop width.
- Verify tables can scroll horizontally when needed, evidence sections remain readable, experiment
choices remain usable, and controls do not overlap or clip.
- Ensure keyboard focus remains visible and all important controls have accessible names.
- Improve the first-time journey only where there is a demonstrated usability issue. Keep the existing
restrained design and do not add decorative charts, generic AI chat, or unnecessary features.

## 4. Audit product reasoning and calculations

Do not loosen existing safeguards to make the UI appear more insightful.

Verify that:
- Summary amounts reconcile with the underlying trade ledger.
- The synthetic-data, simulated-cost, and local-only disclosures remain visible.
- Re-entry comparisons are descriptive, mutually exclusive, and not framed as causal findings.
- Weak or fragile evidence results in a clear abstention.
- Sensitivity analysis never labels a winning trade as the largest losing trade.
- Plan deviations are not described as inherently bad, costly, or causal.
- The app never claims avoidable losses, guaranteed savings, or improved returns.
- The limited-data and empty examples work.
- Selecting, replacing, clearing, and reloading a saved experiment behaves as documented.
- Currency arithmetic, IST date semantics, exclusions, and evidence reconciliation remain covered by
tests.

Fix concrete defects only. Add regression tests for every new code fix. Never weaken a test just to
make it pass.

## 5. Audit the submission PDF and supporting documents
```

```
The required artifacts are:
1. A 300–700-word product one-pager.
2. Actual prompts used, including genuine failed attempts and revisions.
3. Evaluations actually used and how the prototype was judged good enough.
4. A working, accessible app link.

Check that:
- The one-pager is within the required word count and clearly explains the problem, target user, why
Nubra, solution space, scope, and exclusions.
- Product-management reasoning is prominent; technical implementation details support it rather than
dominate it.
- Prompt history contains actual prompts and truthful revisions, not reconstructed or fabricated
conversations.
- The evaluation document distinguishes automated tests, manual checks, and unverified checks.
- The final PDF, README, and evaluation notes agree on the app URL and deployment status.
- No artifact claims interviews, usability research, user demand, financial impact, or public-access
verification that did not happen.
- The walkthrough and submission are consistent with the actual shipped UI.

Avoid rewriting accurate content merely for stylistic reasons.

## 6. Final verification

Run the existing relevant checks and production build. At minimum, run:
- `npm test`
- `npm run test:e2e`
- `npm run build`
- `TEST_PRODUCTION=1 npm run test:e2e`

Report the exact results, including failures. Do not say a command passed unless you ran it.

If you can run browser checks, inspect both the local production build and the actual public URL.
Clearly distinguish these environments. If the public URL is inaccessible to you, state that limitation.

## 7. Deliverables and final report

Make the smallest set of changes needed to reach a submission-ready state.

Return:
1. Blockers, important issues, and minor suggestions found.
2. Files changed and why.
3. Exact tests run and their results.
4. Which public URL is the final intended URL and whether it was independently verified.
5. Outstanding manual checks.
6. The exact path to the updated submission PDF and any other revised artifacts.
7. A concise final GO/NO-GO decision.

Do not fabricate deployment results, make unsupported claims, commit or push without checking the
remote and authorization, or redesign a functioning app merely to make more changes.

Success means a coherent, credible, well-tested product submission—not maximum feature count.
```

10-final-url-clarification.txt

```
sparkling-pixie-91ad77.netlify.app — updated ZIP deployment
```

Actual failed attempts and revisions

The first plan proposed a fourth-entry rule, a 30-minute re-entry cutoff and eight-trade/three-day minima. The independent critique prompt challenged their justification. Those abandoned rules are not in the shipped analysis.

First browser run: 3 passed, 1 failed. The test expected a Unicode minus; the rupee formatter rendered an ASCII minus. The exact amount was correct. The expectation was corrected to the actual formatter output; the next run passed all 4 tests.

Audit regression: all-winning re-entries still generated evidence calling a positive trade the largest losing trade. The engine was changed to run sensitivity only for eligible adverse comparisons.

The first large-sum fixture did not expose rounding. A refined fixture returned 2 instead of the expected 3 paise. Exact BigInt accumulation fixed the defect. Before fixes: 15 passed, 2 failed; afterward: 17 passed.

An ad hoc browser scroll check read scrollLeft before native scrolling completed. The existing Playwright mobile test now polls for scrolling to finish; the application scrolling was unchanged. Final browser suite: 5 passed.

In the final supplemental audit, pointer-mode focus was initially checked as if it were keyboard focus, and valid cached reload responses (304) were initially treated as failures. Keyboard input was simulated correctly, and initial-load 200 checks were separated from cache revalidation. The supplemental audit passed; existing assertions and app logic were unchanged.

How I decided it was good enough

The shipping criteria were exact arithmetic, traceable evidence, refusal to force a weak observation, and a working review-to-experiment journey. Attractive output alone was not a pass condition. These evaluations establish prototype behavior, not trader demand or improved returns.

Automated checks actually executed

Command	Result
npm test	17 passed; 0 failed/skipped
npm run test:e2e	5 Chromium tests passed
npm run build	Production Vite build passed
TEST_PRODUCTION=1 npm run test:e2e	5 tests passed against built bundle

Calculation and evidence checks

Hand-calculated arithmetic: 10 shares bought at ₹100 and sold at ₹110 gives ₹100 gross. Subtracting ₹12 costs gives ₹88 net. Tests also cover missing/zero costs, break-even win-rate denominators, exact duplicates, conflicting IDs, malformed records, overlap rejection and safe integer amounts.

IST close date determines month inclusion; IST entry date determines trading-day sequences and plan counts. Tests exercise month boundaries, equivalent timezone offsets, invalid calendar dates and ambiguous simultaneous exits/entries.

Re-entry and non-re-entry groups are mutually exclusive. Evidence reconciles with displayed totals. Insufficient data and empty periods produce abstention. A largest-loss-removal reversal withholds the card. A profitable plan deviation remains a deviation without a causal claim.

Browser tests cover summary and evidence amounts, experiment selection/reload/replacement/clearing, blocked-storage fallback, limited/empty examples, keyboard focus, skip navigation, 375px overflow checks and horizontal table scrolling.

A supplemental local-production inspection also passed: initial HTML/JavaScript/CSS loaded with 200 responses, reloads used valid 304 cache revalidation, and 1440px/375px evidence and experiment controls were inspected. This was a local preview, not the public Netlify site.

Default synthetic month: reconciliation

12 trades · ₹84 gross − ₹144 modeled costs = −₹60 net · 5 winners / 12 = 41.7%. Four re-entries total −₹720 (mean −₹180); eight non-re-entries total ₹660 (mean ₹82.50). Excluding T2 (−₹240) leaves three re-entries totaling −₹480 (mean −₹160). Four beyond-limit entries total −₹60, including a profitable ₹90 entry. Observation amounts overlap and must not be added as losses.

Hosted checks I performed manually

On the prior Netlify deployment, I captured Chrome Incognito screenshots showing the landing page, summary, both evidence panels, ledger, limited-data abstention and empty state. I confirmed that a selection survived reload, clearing survived reload, and the rightmost evidence columns were reachable at 375px width. These are my manual checks of that prior deployment, not independent automation of the new public URL.

Known limits and unverified checks

No user interviews, usability study, screen-reader test, comprehensive cross-browser test, measured financial benefit or adoption result is claimed. Public JavaScript/CSS HTTP status codes were not independently checked. Fresh curl and Chromium attempts to the confirmed final URL were blocked by the cloud proxy (CONNECT 403 / ERR_TUNNEL_CONNECTION_FAILED); no public page or assets were retrieved. The Netlify badge overlaps some content while scrolling on mobile. The dropdown id/name fix and UI refinements passed local build/browser tests. I then uploaded the updated ZIP to the new Netlify project linked in Artifact 04. Its publication is shown in the deployment screenshot; the complete Incognito journey on that new origin has not been rechecked.

Open the TradeMirror prototype

https://sparkling-pixie-91ad77.netlify.app

Hosting: Netlify · Method: manual static-build upload · The app has no in-app login or brokerage connection. I explicitly confirmed this as the final URL. The updated ZIP upload is candidate-reported and shown as Published in Netlify; independent public page, asset and workflow access remain unverified. Previous Incognito checks apply to the prior deployment.

A short walkthrough

1. Open the full synthetic month and select Review this month. Inspect the factual summary.

2. Expand both evidence panels. Review the exact trade rows, definitions, sensitivity calculation and limitations.

3. Choose one experiment, reload and confirm it remains selected. Clear it and reload again.

4. Switch to the limited-data and empty examples to see the product deliberately withhold unsupported conclusions.

Scope and version note

This is a synthetic take-home prototype, not a production trading tool. It supports normalized, fully closed intraday long-equity positions. It does not import raw broker orders, infer emotions or causation, provide trading advice, or estimate avoidable losses. I report uploading the updated visual-polish ZIP, including the minor form-field fix, to this new Netlify project. Repeat the short walkthrough above in Incognito on this origin; storage is origin-specific, so selections from the old URL will not transfer.

Supporting prompt sources

The 10 preserved prompt/clarification text files are embedded as attachments in this PDF to preserve their exact source text. They are also printed in Artifact 02. Historical URLs in verbatim prompts are retained; the confirmed final submission link is the link in Artifact 04. Attachment availability depends on the PDF viewer; the visible pages contain all four required artifacts.